

# Python Data Analysis Assignment 2 - Project Documentation

## 1. Project Title

### Data Cleaning and Visualization Using Python

## 2. Project Overview

This project focuses on performing basic data analysis, data cleaning, and data visualization using Python.

For this project, the built-in Seaborn `taxis` dataset is used. The dataset contains information about taxi trips, including pickup and dropoff times, passengers, distance, fare, tips, payment methods, and pickup and dropoff boroughs.

The project demonstrates how Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn can be used to clean, analyze, and visualize real-world data.

## 3. Objectives

The main objectives of this project are:

- To load the Seaborn `taxis` dataset
- To explore the structure of the dataset
- To identify missing values
- To handle missing values
- To convert the pickup column into datetime format
- To perform basic data analysis
- To create different types of visualizations
- To analyze the relationship between distance and fare
- To analyze correlations between numerical variables

- To understand basic exploratory data analysis using Python

## 4. Dataset Description

### Dataset Name

Taxis Dataset

### Dataset Source

The dataset is available as a built-in dataset in the Seaborn Python library.

### Dataset Size

- Number of Rows: 6,433
- Number of Columns: 14

### Dataset Columns

| Column     | Description           |
|------------|-----------------------|
| pickup     | Pickup date and time  |
| dropoff    | Dropoff date and time |
| passengers | Number of passengers  |
| distance   | Trip distance         |
| fare       | Trip fare             |
| tip        | Tip amount            |
| tolls      | Toll amount           |
| total      | Total trip amount     |
| color      | Taxi color            |
| payment    | Payment method        |

|                 |                 |
|-----------------|-----------------|
| pickup_zone     | Pickup zone     |
| dropoff_zone    | Dropoff zone    |
| pickup_borough  | Pickup borough  |
| dropoff_borough | Dropoff borough |

## 5. Technologies and Libraries Used

### Programming Language

- Python

### Libraries

- Pandas
- NumPy
- Matplotlib
- Seaborn

### Development Environment

- Jupyter Notebook

## 6. Data Loading

The Seaborn `taxis` dataset was loaded using the following code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("taxis")
```

```
df.head()
```

The `head()` function was used to display the first five rows of the dataset.

## 7. Data Exploration

The structure and information of the dataset were checked using:

```
df.info()
```

This was used to understand:

- Number of rows
- Number of columns
- Column names
- Data types
- Non-null values

## 8. Missing Value Analysis

Missing values were identified using:

```
df.isnull().sum()
```

This helped identify which columns contained missing values.

## 9. Handling Missing Values

Missing values were handled differently for numerical and categorical columns.

### Numerical Columns

Numerical missing values were replaced with the median value.

```
numerical_columns = df.select_dtypes(include=np.number).columns
```

```
for col in numerical_columns:  
    df[col] = df[col].fillna(df[col].median())
```

## Categorical Columns

Categorical missing values were replaced with the mode value.

```
categorical_columns = df.select_dtypes(include="object").columns
```

```
for col in categorical_columns:  
    if df[col].isnull().sum() > 0:  
        df[col] = df[col].fillna(df[col].mode()[0])
```

After handling missing values, the dataset was checked again:

```
df.isnull().sum()
```

The final result showed no missing values in the dataset.

## 10. Datetime Conversion

The pickup column was converted into datetime format.

```
df["pickup"] = pd.to_datetime(df["pickup"])
```

```
df["pickup"].dtype
```

This conversion allows the pickup timestamp to be used effectively for time-based analysis and visualization.

# 11. Data Visualization Using Matplotlib

## 11.1 Line Chart - Fare Over Time

A line chart was created to visualize fare over time.

### Variables Used

- X-axis: Pickup Time
- Y-axis: Fare

### Code

```
df = df.sort_values("pickup")

plt.figure(figsize=(12, 6))

plt.plot(df["pickup"], df["fare"])

plt.title("Fare Over Time")
plt.xlabel("Pickup Time")
plt.ylabel("Fare")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

### Purpose

The line chart helps visualize how taxi fares vary across pickup times.

## 11.2 Bar Chart - Total Fare by Pickup Borough

A bar chart was created to show the total fare collected for each pickup borough.

## Code

```
borough_fare = df.groupby("pickup_borough")["fare"].sum()

plt.figure(figsize=(10, 6))

borough_fare.plot(kind="bar")

plt.title("Total Fare by Pickup Borough")
plt.xlabel("Pickup Borough")
plt.ylabel("Total Fare")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

## Purpose

The chart allows comparison of total fare values across different pickup boroughs.

## 11.3 Pie Chart - Payment Method Distribution

A pie chart was created to visualize the distribution of trips based on payment method.

## Code

```
payment_counts = df["payment"].value_counts()

plt.figure(figsize=(8, 8))

plt.pie(
    payment_counts,
    labels=payment_counts.index,
    autopct="%1.1f%%",
    startangle=90
)
```

```
plt.title("Distribution of Trips by Payment Method")  
plt.show()
```

### **Purpose**

The pie chart shows the proportion of trips made using different payment methods.

## **11.4 Histogram - Trip Distance**

A histogram was created to visualize the distribution of trip distances.

### **Code**

```
plt.figure(figsize=(10, 6))  
  
plt.hist(df["distance"], bins=30)  
  
plt.title("Distribution of Trip Distance")  
plt.xlabel("Distance")  
plt.ylabel("Frequency")  
  
plt.tight_layout()  
plt.show()
```

### **Purpose**

The histogram helps understand the frequency distribution of taxi trip distances.

## **11.5 Box Plot - Tips by Pickup Borough**

A box plot was created to visualize the distribution of tips for each pickup borough.

## Code

```
plt.figure(figsize=(12, 6))

df.boxplot(
    column="tip",
    by="pickup_borough"
)

plt.title("Distribution of Tips by Pickup Borough")
plt.suptitle("")
plt.xlabel("Pickup Borough")
plt.ylabel("Tip")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

## Purpose

The box plot helps compare tip distributions and identify variation and potential outliers across pickup boroughs.

# 12. Data Visualization Using Seaborn

## 12.1 Count Plot - Trips by Pickup Borough

A count plot was created to visualize the number of trips in each pickup borough.

## Code

```
plt.figure(figsize=(10, 6))

sns.countplot(
    data=df,
    x="pickup_borough"
```

```
)

plt.title("Number of Trips by Pickup Borough")
plt.xlabel("Pickup Borough")
plt.ylabel("Number of Trips")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

## Purpose

The count plot shows the number of taxi trips recorded for each pickup borough.

## 12.2 Scatter Plot - Distance vs Fare

A scatter plot was created to analyze the relationship between trip distance and fare.

### Variables Used

- X-axis: Distance
- Y-axis: Fare
- Hue: Pickup Borough

### Code

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="distance",
    y="fare",
    hue="pickup_borough"
)

plt.title("Relationship Between Distance and Fare")
plt.xlabel("Distance")
```

```
plt.ylabel("Fare")
```

```
plt.tight_layout()  
plt.show()
```

## Purpose

The scatter plot helps visualize how fare values vary with trip distance and allows comparison across pickup boroughs.

## 12.3 Correlation Heatmap

A correlation heatmap was created to analyze the relationship between numerical variables.

### Variables Used

- Distance
- Fare
- Tip
- Tolls
- Total

### Code

```
numeric_data = df[  
    ["distance", "fare", "tip", "tolls", "total"]  
]
```

```
correlation = numeric_data.corr()
```

```
plt.figure(figsize=(10, 7))
```

```
sns.heatmap(  
    correlation,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"
```

```
)  
  
plt.title("Correlation Heatmap")  
plt.tight_layout()  
plt.show()
```

## Purpose

The heatmap provides a visual representation of the correlation between numerical variables.

# 13. Final Dataset Validation

After completing the data cleaning and analysis, the final dataset was checked.

## Code

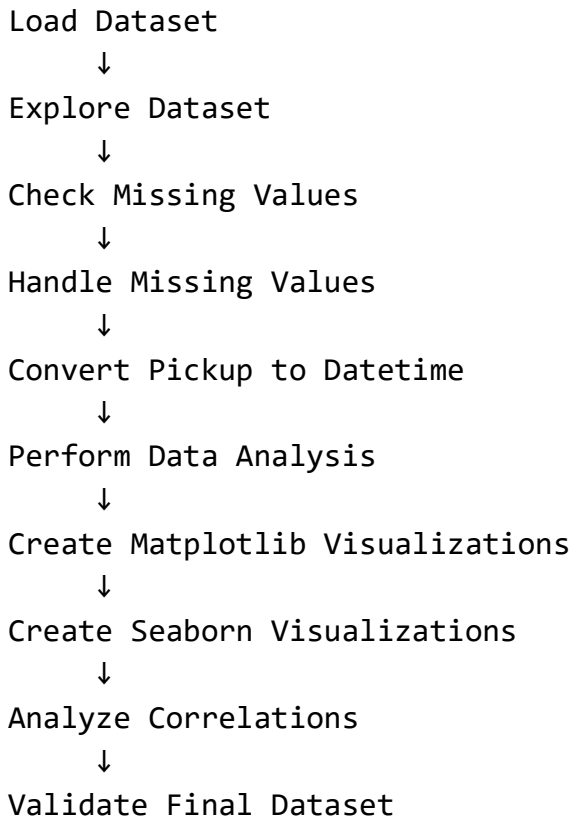
```
print("Dataset Shape:")  
print(df.shape)  
  
print("\nMissing Values:")  
print(df.isnull().sum())  
  
print("\nFirst 5 Rows:")  
display(df.head())
```

## Final Dataset Result

- Rows: 6,433
- Columns: 14
- Missing values: 0 after data cleaning

## 14. Project Workflow

The complete workflow followed in this project is:



## 15. Key Analysis Areas

The project covers the following analysis areas:

### **Fare Analysis**

The total fare was analyzed based on pickup borough.

### **Payment Analysis**

The distribution of taxi trips across different payment methods was visualized.

## **Distance Analysis**

The distribution of trip distances was analyzed using a histogram.

## **Tip Analysis**

Tip distributions across pickup boroughs were analyzed using a box plot.

## **Trip Analysis**

The number of trips across pickup boroughs was visualized using a count plot.

## **Relationship Analysis**

The relationship between distance and fare was visualized using a scatter plot.

## **Correlation Analysis**

The correlation between distance, fare, tip, tolls, and total was analyzed using a heatmap.

# **16. Learning Outcomes**

After completing this project, I learned:

- How to load a dataset using Seaborn
- How to inspect a dataset using Pandas
- How to identify missing values
- How to handle numerical missing values
- How to handle categorical missing values
- How to convert columns into datetime format
- How to use Pandas for data manipulation
- How to use NumPy for numerical operations
- How to create visualizations using Matplotlib
- How to create visualizations using Seaborn
- How to create line charts
- How to create bar charts
- How to create pie charts

- How to create histograms
- How to create box plots
- How to create count plots
- How to create scatter plots
- How to create correlation heatmaps
- How to perform basic exploratory data analysis

## 17. Project Files

The project contains the following files:

```
Python-DA-Assignment-2/  
├── Python_DA_Assignment_2_Data_Visualization.ipynb  
├── README.md  
└── PROJECT_DOCUMENTATION.md
```

## 18. Conclusion

This project demonstrates the complete basic workflow of data cleaning, analysis, and visualization using Python.

The Seaborn `taxi` dataset was explored and cleaned by handling missing values and converting the `pickup` column into datetime format.

Different visualizations were created using Matplotlib and Seaborn to understand fare, distance, tips, payment methods, pickup boroughs, and relationships between numerical variables.

This project helped strengthen my practical understanding of Python-based data analysis and data visualization.

## **19. Author**

**Hemarubini**

**Python Data Analytics Learner**

## **20. Skills Demonstrated**

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Data Cleaning
- Data Analysis
- Data Visualization
- Exploratory Data Analysis
- Jupyter Notebook